

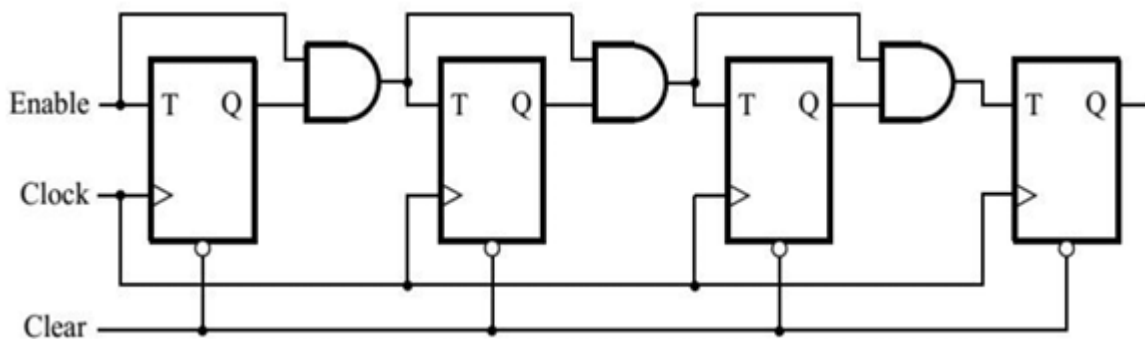
BÁO CÁO THIẾT KẾ HỆ THỐNG SỐ VỚI HDL - LỚP CE213.Q23.2

Bài thực hành số 2

Giảng viên hướng dẫn	Hồ Ngọc Diễm		Điểm
Sinh viên thực hiện	Vũ Đại Dương	23520359	
	Nguyễn Đặng Phương Duy	23520372	

Câu 1:

1.1. Thiết kế bộ đếm 4 bit như hình:



Hình 1 Mạch yêu cầu thiết kế

- Thiết kế T-FF:

```
module t_ff (  
    input wire t, clk, clear_n,  
    output reg q  
);  
  
always @(posedge clk or negedge clear_n) begin  
    if (clear_n == 0) begin  
        q <= 1'b0;  
    end  
    else if (t) begin  
        q <= ~q;  
    end  
end  
endmodule
```

Hình 2 Code verilog khối T-FlipFlop

- Thiết kế bộ đếm 4 bit:

```

module counter_4bit_const (
    input wire enable, clock, clear,
    output wire [3:0] q
);
    wire t1, t2, t3;

    assign t1 = q[0] & enable;
    assign t2 = q[1] & t1;
    assign t3 = q[2] & t2;

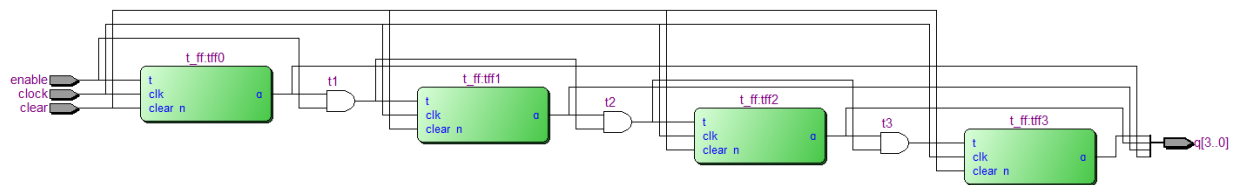
    t_ff tff0 (enable, clock, clear, q[0]);
    t_ff tff1 (t1, clock, clear, q[1]);
    t_ff tff2 (t2, clock, clear, q[2]);
    t_ff tff3 (t3, clock, clear, q[3]);

endmodule

```

Hình 3 Code verilog thiết kế theo yêu cầu

- RTL Viewer:



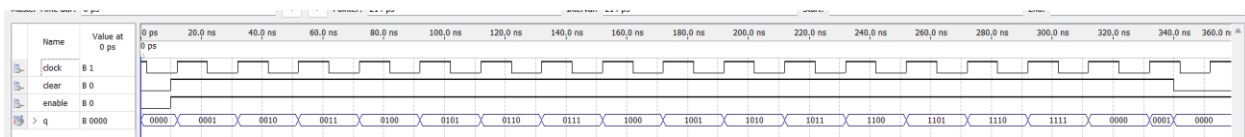
Hình 4 RTL của thiết kế

- Tần số Fmax của mạch:

	Fmax	Restricted Fmax	Clock Name	Note
1	715.31 MHz	420.17 MHz	clock	limit due to minimum period restriction (max I/O toggle rate)

Hình 5 Fmax của thiết kế

- Waveform:



Hình 6 Waveform thiết kế

⇒ Waveform đúng với mong đợi

- Clear = 0: thực hiện reset bất đồng bộ
- Clear = 1 & en = 1: thực hiện đếm từ 0000 đến 1111

1.2. Thiết kế bộ đếm 4 bit được mô tả ở mức Behavior

- Thiết kế bộ đếm:

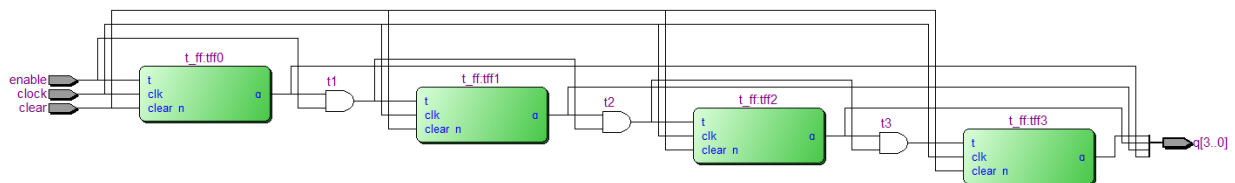
```
module counter_4bit_behav (  
    input wire enable, clock, clear,  
    output reg [3:0] q  
);  
  
always @(posedge clock or negedge clear) begin  
    if (clear == 0) begin  
        q <= 4'd0;  
    end  
    else if (enable) begin  
        q <= q + 1;  
    end  
end  
endmodule
```

Hình 7 Code verilog theo yêu cầu

- So sánh RTL và Fmax của mạch 1.1 và 1.2

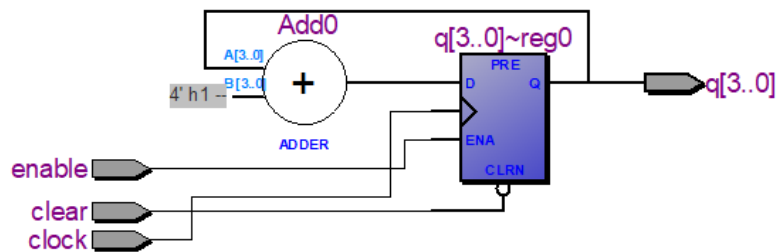
- o RTL:

- 1.1:



Hình 8 RTL của thiết kế 1.1

- 1.2:



Hình 9 RTL của thiết kế 1.2

- o Fmax:

- 1.1:

	Fmax	Restricted Fmax	Clock Name	Note
1	715.31 MHz	420.17 MHz	clock	limit due to minimum period restriction (max I/O toggle rate)

Hình 10 Fmax của thiết kế 1.1

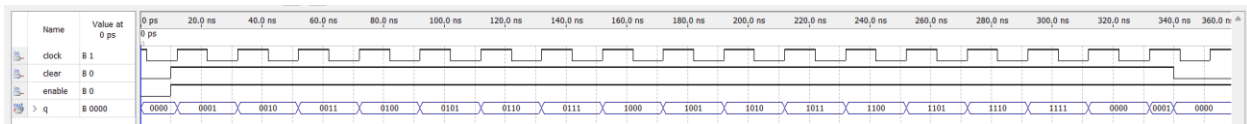
■ 1.2:

	Fmax	Restricted Fmax	Clock Name	Note
1	715.31 MHz	420.17 MHz	clock	limit due to minimum period restriction (max I/O toggle rate)

Hình 11 Fmax của thiết kế 1.2

⇒ Có thể thấy mạch 1.2 được Quartus tối ưu hơn so với mạch 1.1 tự thiết kế thủ công. Tuy nhiên 2 thiết kế vẫn đạt Fmax bằng nhau.

- Waveform:

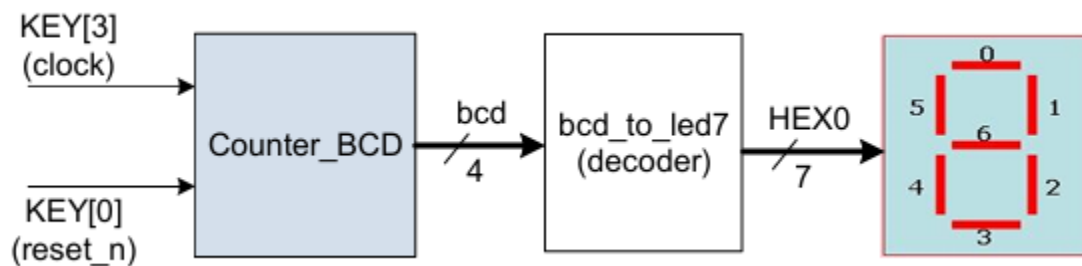


Hình 12 Waveform của thiết kế 1.2

⇒ Waveform đã chạy đúng với mong đợi

Câu 2:

2.1. Thiết kế bộ đếm BCD tăng theo cạnh lên xung clock



Hình 13 Mạch yêu cầu thiết kế

- Thiết kế BCD_Counter:

```

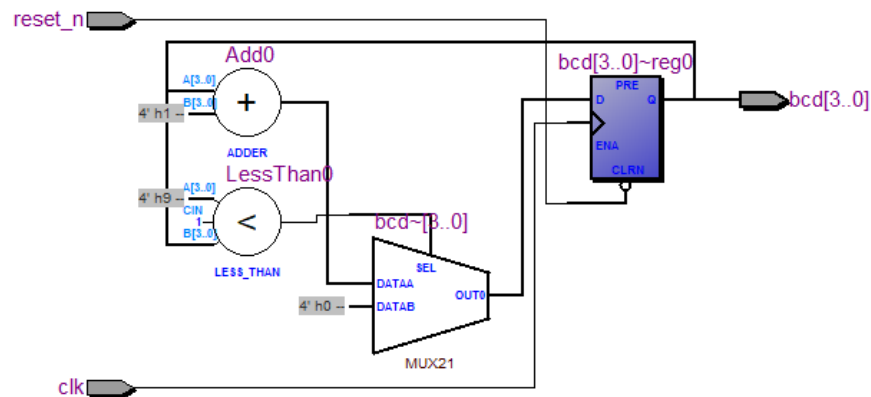
module BCD_counter (
    input wire clk, reset_n,
    output reg [3:0] bcd
);

always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        bcd <= 4'b0000;
    end
    else begin
        if (bcd >= 4'b1001) begin
            bcd <= 4'b0000;
        end
        else begin
            bcd <= bcd + 1'b1;
        end
    end
end
endmodule

```

Hình 14 Code verilog khối BCD counter

○ RTL Viewer:



Hình 15 RTL khối BCD counter

- Thiết kế decoder 7 segments:

```

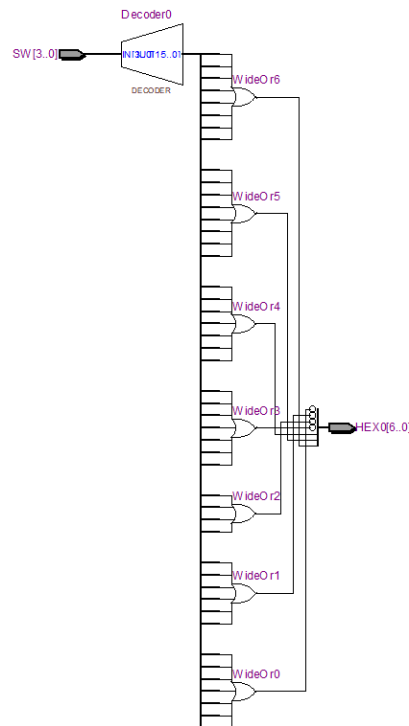
module bcd_4bit_to_led7 (
    input wire [3:0] SW,
    output reg [6:0] HEX0
);

    always @(*) begin
        case (SW)
            4'b0000: HEX0 = 7'b1000000;
            4'b0001: HEX0 = 7'b1111001;
            4'b0010: HEX0 = 7'b0100100;
            4'b0011: HEX0 = 7'b0110000;
            4'b0100: HEX0 = 7'b0011001;
            4'b0101: HEX0 = 7'b0010010;
            4'b0110: HEX0 = 7'b0000010;
            4'b0111: HEX0 = 7'b1111000;
            4'b1000: HEX0 = 7'b0000000;
            4'b1001: HEX0 = 7'b0010000;
            default: HEX0 = 7'b1111111;
        endcase
    end
endmodule

```

Hình 16 Code verilog khối giải mã LED 7 đoạn

○ RTL Viewer:



Hình 17 RTL của decoder 7 segments

- Thiết kế bộ đếm BCD:

```

module BCD_counter_posedge_clk (
    input wire    [3:0]    KEY,
    output wire   [6:0]    HEX0
);

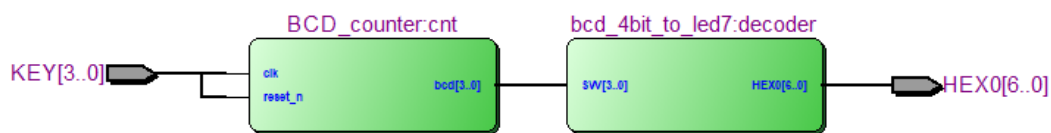
    wire [3:0] bcd;
    BCD_counter cnt (KEY[3], KEY[0], bcd);
    bcd_4bit_to_led7 decoder (bcd, HEX0);

endmodule

```

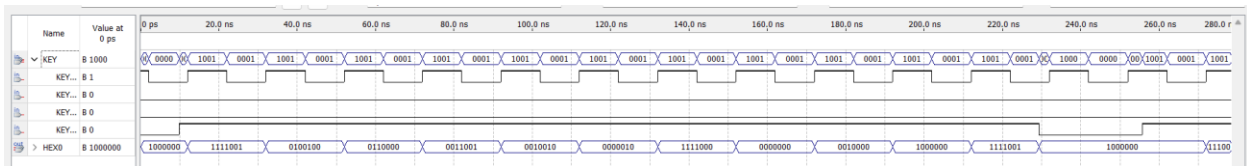
Hình 18 Code verilog theo yêu cầu 2.1

- RTL Viewer:



Hình 19 RTL của thiết kế

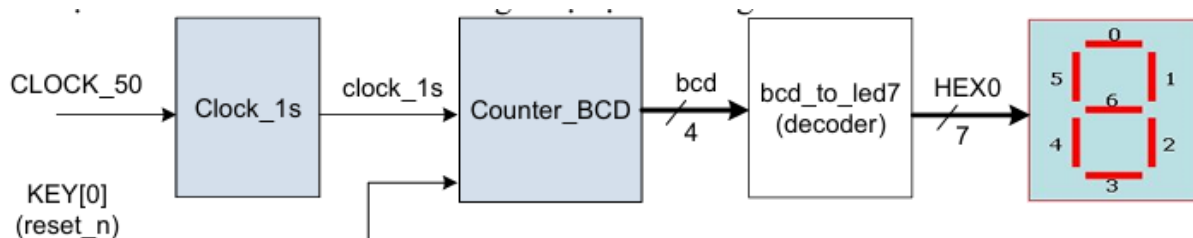
- Waveform:



Hình 20 Waveform của thiết kế

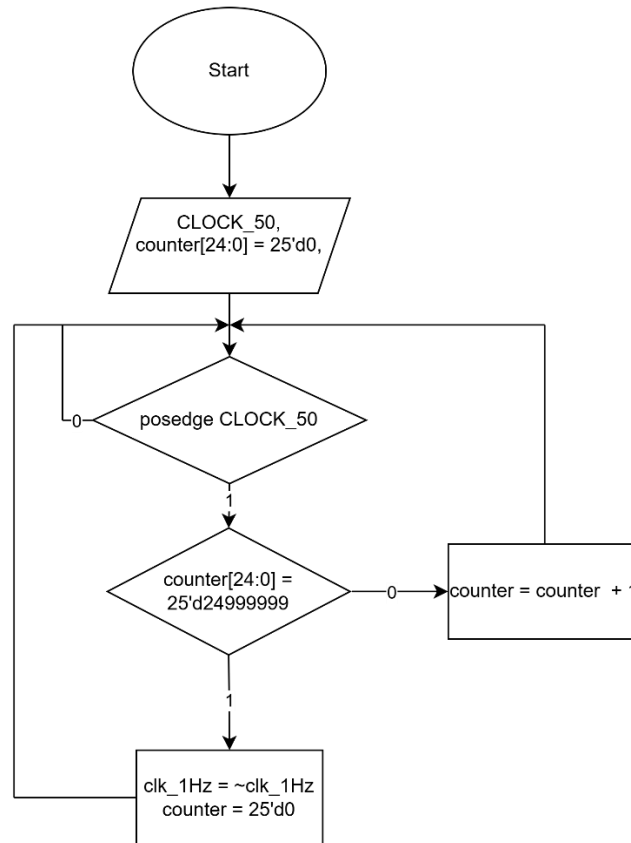
⇒ Kết quả waveform đúng với kết quả mong đợi

2.2. Thiết kế bộ đếm BCD tăng mỗi 1s



Hình 21 Mạch yêu cầu thiết kế

- Lưu đồ giải thuật khối Clock_1s:



Hình 22 Lưu đồ giải thuật của khối Clock 1s

- Thiết kế khối Clock_1s:

```

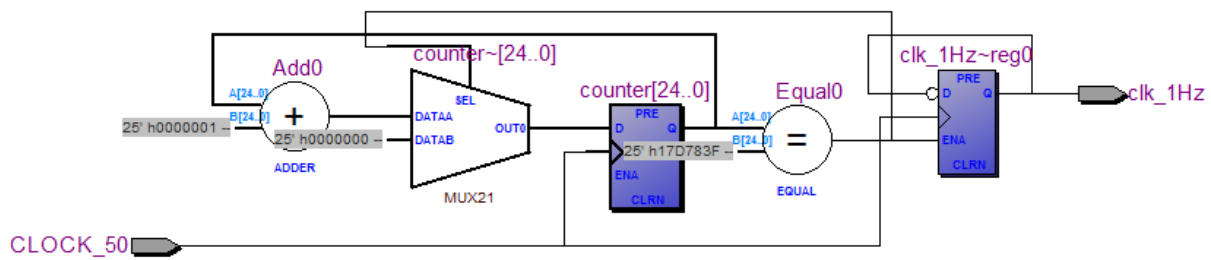
module clock_1s (
    input wire CLOCK_50,
    output reg clk_1Hz
);
    reg [24:0] counter = 25'd0;

    always @(posedge CLOCK_50) begin
        if (counter == 25'd24_999_999) begin
            counter <= 25'd0;
            clk_1Hz <= ~clk_1Hz;
        end
        else begin
            counter <= counter + 1'b1;
        end
    end
endmodule

```

Hình 23 Code verilog khối clock_1s

- RTL Viewer:



Hình 24 RTL khối clock_1s

- Thiết kế bộ đếm tăng mỗi 1s:

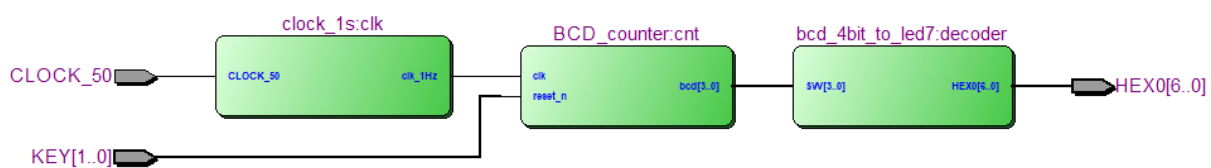
```
module BCD_counter_1s (
    input wire CLOCK_50,
    input wire [1:0] KEY,
    output wire [6:0] HEX0
);

    wire clk_1s;
    wire [3:0] bcd;

    clock_1s clk (CLOCK_50, clk_1s);
    BCD_counter cnt (clk_1s, KEY[0], bcd);
    bcd_4bit_to_led7 decoder (bcd, HEX0);
endmodule
```

Hình 25 Code theo yêu cầu 2.2

- RTL Viewer:



Hình 26 RTL của thiết kế

- Kiểm tra thiết kế (*)
- 2.3. Thiết kế bộ đếm BCD 2 digit từ 00 đến 20, giá trị tăng mỗi 1s
- Thiết kế khối BCD_counter 2 digit từ 00 đến 20 tăng theo xung clk:

- ⇒ Waveform đã chạy đúng theo kết quả mong đợi
- Reset_n = 1 → Reset về 00
 - Reset_n = 0 → Giá trị tăng theo cạnh lên xung clk từ 00 đến 20
- Thiết kế bộ đếm BCD từ 00 đến 20 tăng mỗi 1s:

```

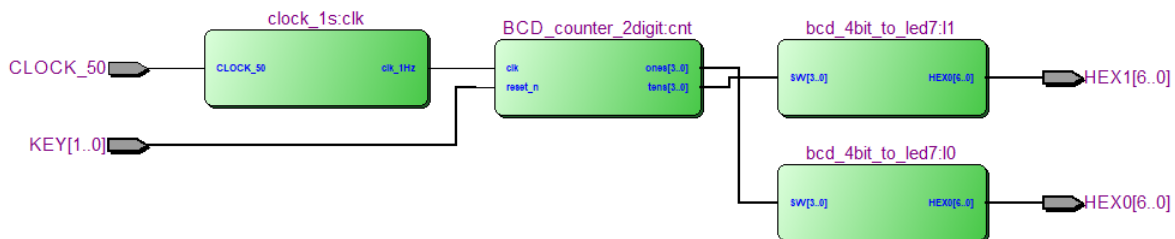
module BCD_counter_2digit_1s (
    input wire CLOCK_50,
    input wire [1:0] KEY,
    output wire [6:0] HEX0, HEX1
);
    wire clk_1s;
    wire [3:0] ones, tens;

    clock_1s clk (CLOCK_50, clk_1s);
    BCD_counter_2digit cnt (clk_1s, KEY[0], ones, tens);
    bcd_4bit_to_led7 10 (ones, HEX0);
    bcd_4bit_to_led7 11 (tens, HEX1);
endmodule

```

Hình 30 Code verilog theo yêu cầu 2.3

- RTL Viewer:



Hình 31 RTL của thiết kế

- Kiểm tra thiết kế (*)

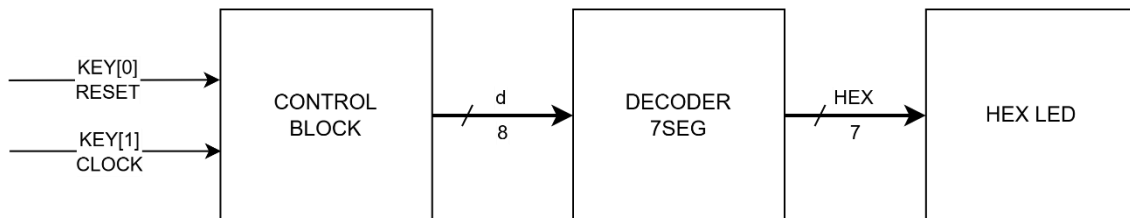
Câu 3:

3.1. Thực hiện bảng chạy chữ như mô tả ở hình dưới. Sử dụng HEX7-0 để hiển thị kí tự.

Clock cycle	Character pattern						
	H7	H6	H5	H4	H3	H2	H1 H0
0				H	E	L	L O
1			H	E	L	L	O
2		H	E	L	L	O	
3	H	E	L	L	O		
4	E	L	L	O			H
5	L	L	O			H	E
6	L	O			H	E	L
7	O			H	E	L	L
8				H	E	L	L O
...	and so on						

Hình 32: Hình mô tả logic chạy chữ

a. Sơ đồ kết nối của mạch



Hình 33: Sơ đồ kết nối của mạch

Bao gồm 2 module và các hex led, trong đó:

- Control block: chứa logic điều khiển mạch theo như hình mô tả
 - o Input: clock và reset
 - o Output: [7:0] d là chữ cái cần hiển thị
 - Decoder 7seg: mạch giải mã chữ cái sang tín hiệu led 7 đoạn
- b. Code verilog thiết kế của mạch
- Khởi control block (hex_shift):

```

1  module hex_shift(
2      input wire clk, rst,
3      output reg [7:0] d0, d1, d2, d3, d4, d5, d6, d7
4  );
5      reg [7:0] display [7:0];
6      reg [7:0] temp;
7      integer i;
8      always @(posedge clk or posedge rst) begin
9          if(rst) begin
10             display[0] = "0";
11             display[1] = "1";
12             display[2] = "1";
13             display[3] = "E";
14             display[4] = "H";
15             display[5] = " ";
16             display[6] = " ";
17             display[7] = " ";
18         end
19         else begin
20             temp = display[7];
21             for (i = 7; i > 0; i = i - 1) begin
22                 display[i] <= display[i-1];
23             end
24             display[0] <= temp;
25         end
26     end
27
28     always @(*) begin
29         d0 = display[0];
30         d1 = display[1];
31         d2 = display[2];
32         d3 = display[3];
33         d4 = display[4];
34         d5 = display[5];
35         d6 = display[6];
36         d7 = display[7];
37     end
38 endmodule

```

Hình 34: Khối control block

- Khối decoder 7seg:

```

1  module decoder_7seg(
2      input [7:0] i,
3      output reg [6:0] o
4  );
5
6      // H 7'b00001001
7      // E 7'b00000110
8      // L 7'b1000111
9      // O 7'b1000000
10
11     always @(*) begin
12         case(i)
13             "H": o = 7'b00001001;
14             "E": o = 7'b00000110;
15             "L": o = 7'b1000111;
16             "O": o = 7'b1000000;
17             " ": o = 7'b1111111;
18             default: o = 7'b1111111;
19         endcase
20     end
21 endmodule

```

Hình 35: Khối decoder_7seg

- Top module:

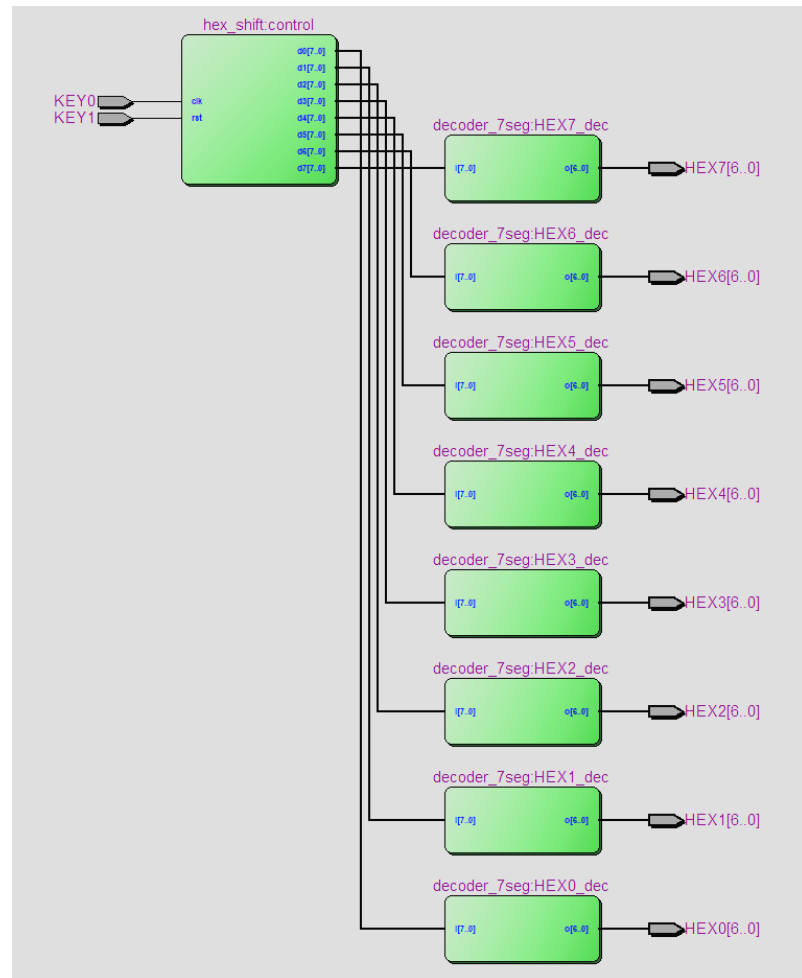
```

1  module bai31(
2      input wire KEY0, KEY1,
3      output wire [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7
4  );
5
6      wire [7:0] temp [7:0];
7
8      hex_shift control (
9          .clk(KEY0),
10         .rst(KEY1),
11         .d0(temp[0]),
12         .d1(temp[1]),
13         .d2(temp[2]),
14         .d3(temp[3]),
15         .d4(temp[4]),
16         .d5(temp[5]),
17         .d6(temp[6]),
18         .d7(temp[7])
19     );
20
21     decoder_7seg HEX0_dec (.i(temp[0]), .o(HEX0));
22     decoder_7seg HEX1_dec (.i(temp[1]), .o(HEX1));
23     decoder_7seg HEX2_dec (.i(temp[2]), .o(HEX2));
24     decoder_7seg HEX3_dec (.i(temp[3]), .o(HEX3));
25     decoder_7seg HEX4_dec (.i(temp[4]), .o(HEX4));
26     decoder_7seg HEX5_dec (.i(temp[5]), .o(HEX5));
27     decoder_7seg HEX6_dec (.i(temp[6]), .o(HEX6));
28     decoder_7seg HEX7_dec (.i(temp[7]), .o(HEX7));
29
30 endmodule

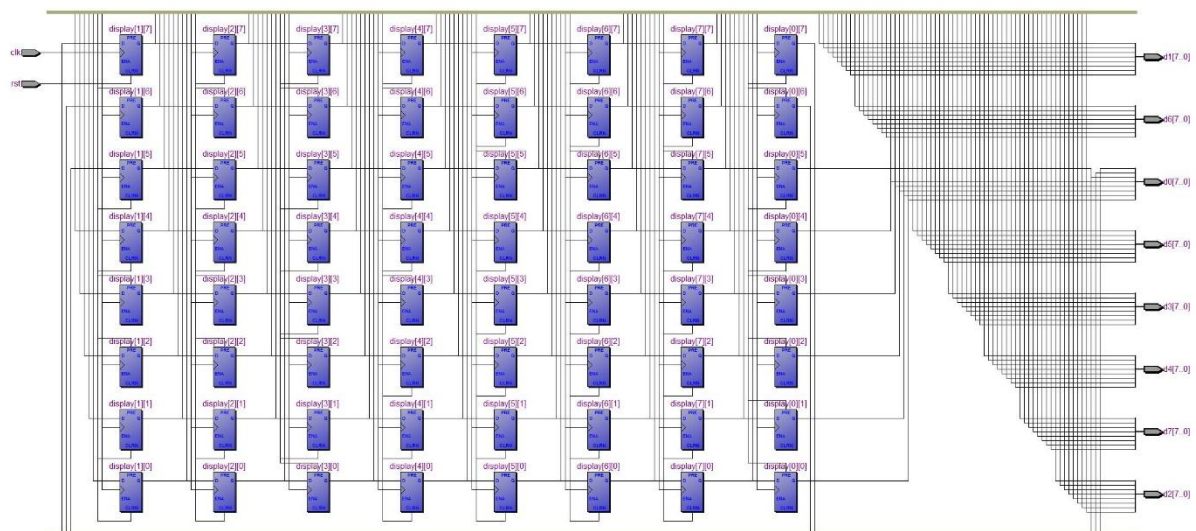
```

Hình 36: Khối top module

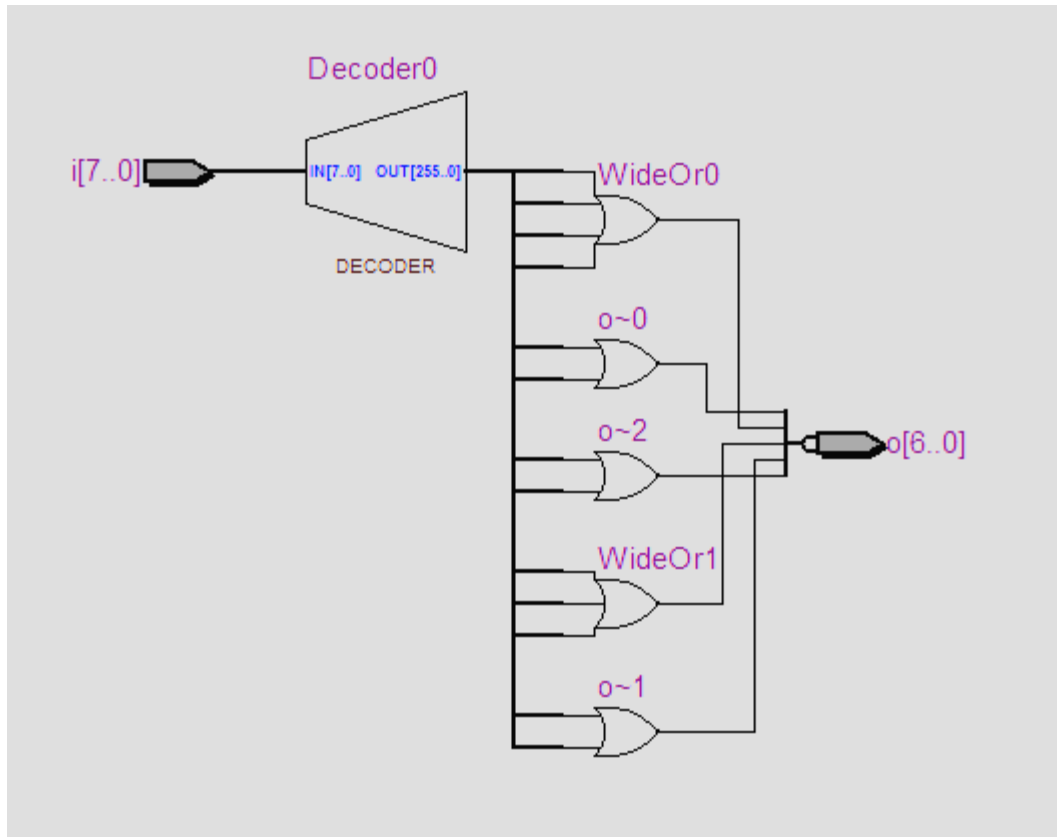
c. RTL view



Hình 37: RTL của thiết kế



Hình 38: RTL khối hex_shift



Hình 39: RTL khối decoder_7seg

d. Kiểm tra thiết kế

3.2. Thực hiện lại 3.1 nhưng điều khiển mạch sao cho các HEX sẽ tự động dịch chuyển sau mỗi khoảng thời gian 1s

Để các HEX tự động dịch chuyển sau mỗi khoảng thời gian 1s thì cần thiết kế khối tạo xung clock sau 1s

- Khối onesec_clk:


```

1  module onsec_clk(
2      input clk, rst,
3      output reg clk_1s
4  );
5
6      reg [25:0] counter;
7
8      always @(posedge clk or posedge rst) begin
9          if (rst) begin
10             clk_1s <= 0;
11             counter <= 0;
12         end else if (counter == 50000000 - 1) begin
13             clk_1s <= 1;
14             counter <= 0;
15         end else begin
16             clk_1s <= 0;
17             counter <= counter + 1;
18         end
19     end
20
21 endmodule

```

Hình 40: Khối onsec_clk

- Khối top module cho bài 3.2

```

1 module bai32(
2     input CLOCK_50,
3     input wire KEY1,
4     output wire [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7
5 );
6
7     wire [7:0] temp [7:0];
8     wire clock;
9
10    onsec_clk onsec (
11        .clk(CLOCK_50),
12        .rst(KEY1),
13        .clk_ls(clock)
14    );
15
16    hex_shift control (
17        .clk(clock),
18        .rst(KEY1),
19        .d0(temp[0]),
20        .d1(temp[1]),
21        .d2(temp[2]),
22        .d3(temp[3]),
23        .d4(temp[4]),
24        .d5(temp[5]),
25        .d6(temp[6]),
26        .d7(temp[7])
27    );
28
29    decoder_7seg HEX0_dec (.i(temp[0]), .o(HEX0));
30    decoder_7seg HEX1_dec (.i(temp[1]), .o(HEX1));
31    decoder_7seg HEX2_dec (.i(temp[2]), .o(HEX2));
32    decoder_7seg HEX3_dec (.i(temp[3]), .o(HEX3));
33    decoder_7seg HEX4_dec (.i(temp[4]), .o(HEX4));
34    decoder_7seg HEX5_dec (.i(temp[5]), .o(HEX5));
35    decoder_7seg HEX6_dec (.i(temp[6]), .o(HEX6));
36    decoder_7seg HEX7_dec (.i(temp[7]), .o(HEX7));
37
38 endmodule

```

Hình 41: Top module bài 3.2

Vì không cần KEY[0] làm xung clock nên loại bỏ input KEY[0] và sử dụng khối onsec_clk làm xung clock, phần còn lại thực hiện giống bài 3.1

Câu 4: Thiết kế thiết kế bộ ALU 32-bit có các chức năng như hình bên dưới. Các chức năng cộng trừ, tăng giảm thực hiện trên các số có dấu bù 2 32-bit. Cờ báo add_sub_overflow sẽ được bật lên 1 khi mạch phát hiện có overflow xảy ra.

M	S _i	S ₀	ALU Operations
0	0	0	Complement A
0	0	1	AND
0	1	0	EX-OR
0	1	1	OR
1	0	0	Decrement A
1	0	1	Add
1	1	0	Subtract
1	1	1	Increment A

a. Code verilog thiết kế của mạch

```
1 module bai4(  
2     input wire [31:0] A, B,  
3     input wire M,  
4     input wire [1:0] S,  
5     output reg [31:0] Y,  
6     output reg add_sub_overflow  
7 );  
8  
9     localparam max_int = 32'h7FFFFFFF;  
10    localparam min_int = 32'h80000000;  
11  
12    always@(*) begin  
13        add_sub_overflow = 0;  
14        if(M) begin  
15            case(S)  
16            2'b00: begin  
17                Y = A - 1;  
18                add_sub_overflow = (Y == max_int);  
19            end  
20            2'b01: begin  
21                Y = A + B;  
22                add_sub_overflow = (~A[31] & ~B[31] & Y[31]) | (A[31] & B[31] & ~Y[31]);  
23            end  
24            2'b10: begin  
25                Y = A - B;  
26                add_sub_overflow = (~A[31] & B[31] & Y[31]) | (A[31] & ~B[31] & ~Y[31]);  
27            end  
28            2'b11: begin  
29                Y = A + 1;  
30                add_sub_overflow = (Y == min_int);  
31            end  
32            endcase  
33        end  
34        else begin  
35            case(S)  
36            2'b00: Y = ~A;  
37            2'b01: Y = A & B;  
38            2'b10: Y = A ^ B;  
39            2'b11: Y = A | B;  
40            endcase  
41            add_sub_overflow = 0;  
42        end  
43    end  
44  
45 endmodule
```

Hình 42: Code verilog bài 4

Phần xử lý add_sub_overflow được thực hiện theo logic sau :

- A+B : overflow khi A và B cùng dấu nhưng kết quả khác dấu
- A-B : overflow khi A và B khác dấu và kết quả khác dấu A
- A-1 : overflow khi kết quả bằng max_int (32'h7FFFFFFF)
- A+1: overflow khi kết quả bằng min_int (32'h80000000)

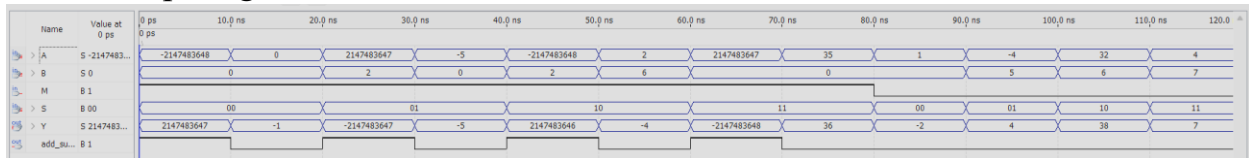
b. RTL view



Hình 43: RTL view của mạch

- Tổng quan RTL của mạch chỉ bao gồm các mux và các cổng logic chứ không có các phần tử nhớ

c. Mô phỏng waveform:

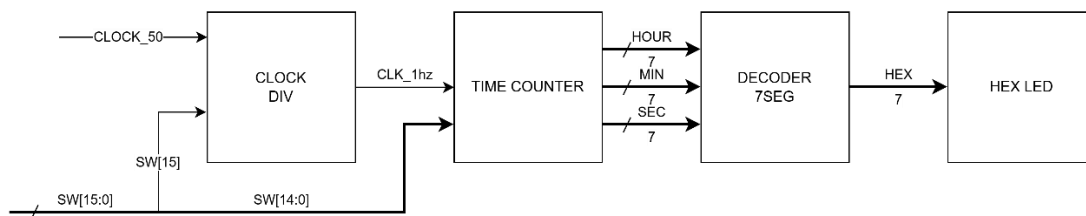


Hình 44: Waveform bài 4

- Kết luận: waveform chạy đúng với logic của mạch

Câu 5: Hiện thực một đồng hồ hiển thị giờ, phút, giây trong ngày. Đồng hồ sẽ thể hiện giá trị “giờ” (từ 0 đến 23) lên các led 7-đoạn HEX7-6, giá trị “phút” (từ 0 đến 60) lên các led HEX5-4, và giá trị “giây” (từ 0 đến 60) lên các led HEX3-2. Sử dụng các SW15-0 để reset lại giá trị “giờ” và “phút” cho đồng hồ. Mạch hiện thực phải có khả năng báo lỗi hoặc không cho thiết lập các giá trị giờ, phút, giây bất hợp lý.

a. Sơ đồ khối của mạch:



Mạch bao gồm:

- CLOCK_DIV: Module tạo xung clock 1hz để phục vụ cho việc đếm giây

- TIME_COUNTER: Module xử lý logic thời gian, thiết lập thời gian từ switch và xử lý báo lỗi nếu thời gian không hợp lý
- DECODER_7SEG: Module giải mã chữ số thành tín hiệu điều khiển led 7 đoạn

b. Code verilog hiện thực mạch

```

1  module clk_div(
2      input wire clk, rst,
3      output reg clk_1hz
4  );
5
6      reg [25:0] counter;
7      always@(posedge rst or posedge clk) begin
8          if(rst) begin
9              counter <= 0;
10             clk_1hz <= 0;
11         end
12         else if(counter == 24999999) begin
13             counter = 0;
14             clk_1hz <= ~clk_1hz;
15         end else
16             counter <= counter + 25'd1;
17     end
18
19 endmodule

```

Hình 45: Module tạo xung clock

```

1  module time_counter(
2      input wire clk, rst, set,
3      input wire [4:0] set_hour,
4      input wire [5:0] set_min,
5      output reg [4:0] hour,
6      output reg [5:0] min,
7      output reg [5:0] sec,
8      output error
9  );
10
11      assign error = (set && (set_hour > 23 || set_min > 59));
12
13      always@(posedge rst or posedge clk) begin
14          if(rst) begin
15              hour <= 5'd0;
16              min <= 6'd0;
17              sec <= 6'd0;
18          end
19          else if(set && !error) begin
20              hour <= set_hour;
21              min <= set_min;
22          end
23          else if(sec == 59) begin
24              sec <= 6'd0;
25              if(min == 59) begin
26                  min <= 6'd0;
27                  if(hour == 23) begin
28                      hour <= 5'd0;
29                  end
30                  else
31                      hour <= hour + 5'd1;
32              end
33              else
34                  min <= min + 6'd1;
35          end
36          else
37              sec <= sec + 6'd1;
38      end
39  endmodule

```

Hình 46: Module thực hiện logic đếm thời gian và báo lỗi

```

1  module decoder_7seg(
2      input wire [3:0] digit,
3      output reg [6:0] seg
4  );
5
6      always @(*) begin
7          case(digit)
8              4'd0: seg = 7'b1000000;
9              4'd1: seg = 7'b1111001;
10             4'd2: seg = 7'b0100100;
11             4'd3: seg = 7'b1001111;
12             4'd4: seg = 7'b0011001;
13             4'd5: seg = 7'b0010010;
14             4'd6: seg = 7'b0000010;
15             4'd7: seg = 7'b1111000;
16             4'd8: seg = 7'b0000000;
17             4'd9: seg = 7'b0010000;
18             default: seg = 7'b1111111;
19         endcase
20     end
21 endmodule

```

Hình 47: Module decoder_7seg

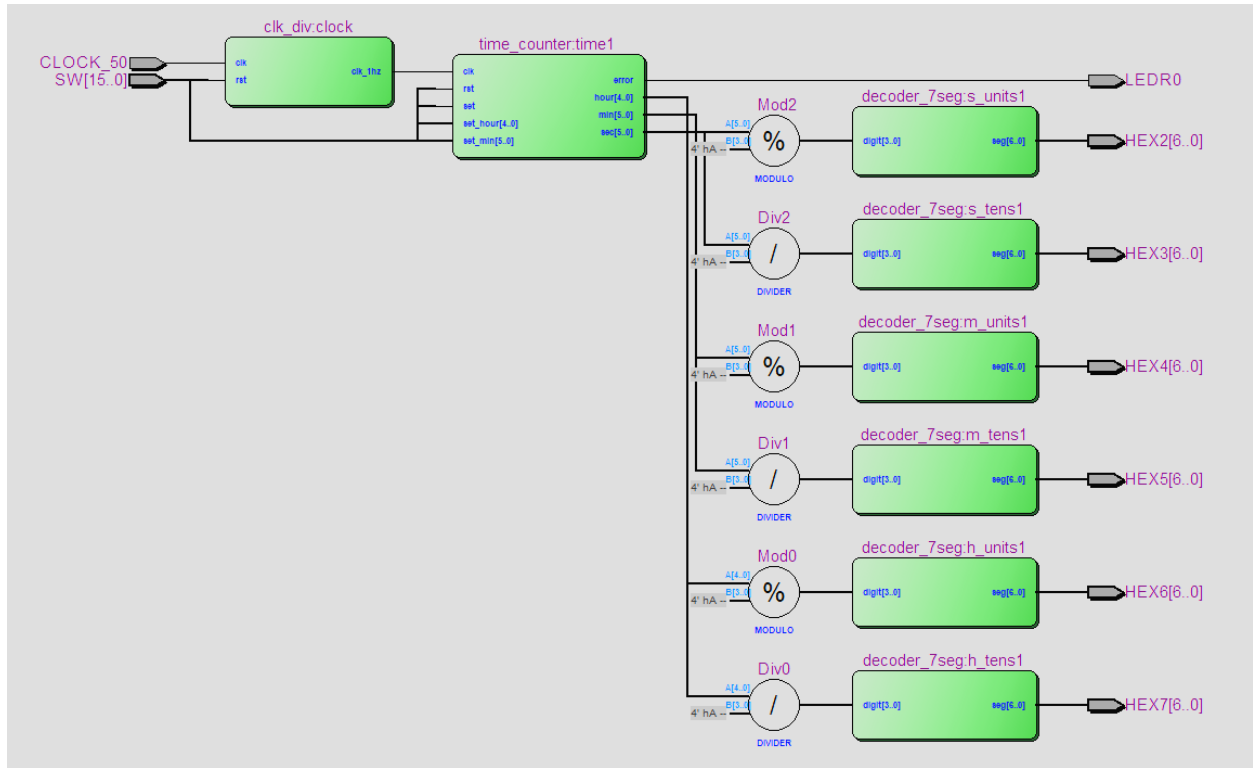
```

1 module bai5(
2     input CLOCK_50,
3     input [15:0] SW,
4     output [6:0] HEX2, HEX3, HEX4, HEX5, HEX6, HEX7,
5     output LEDR0
6 );
7
8     wire clk;
9     clk_div clock (.clk(CLOCK_50), .rst(SW[15]), .clk_1hz(clk));
10
11     wire [4:0] h;
12     wire [5:0] m, s;
13     time_counter time1 (
14         .clk(clk),
15         .rst(SW[15]),
16         .set(SW[14]),
17         .set_hour(SW[13:9]),
18         .set_min(SW[8:3]),
19         .hour(h),
20         .min(m),
21         .sec(s),
22         .error(LEDR0)
23     );
24
25     wire [3:0] h_tens, h_units;
26     assign h_tens = h / 5'd10;
27     assign h_units = h % 5'd10;
28
29     wire [3:0] m_tens, m_units;
30     assign m_tens = m / 6'd10;
31     assign m_units = m % 6'd10;
32
33     wire [3:0] s_tens, s_units;
34     assign s_tens = s / 6'd10;
35     assign s_units = s % 6'd10;
36
37     decoder_7seg h_tens1 (.digit(h_tens), .seg(HEX7));
38     decoder_7seg h_units1 (.digit(h_units), .seg(HEX6));
39     decoder_7seg m_tens1 (.digit(m_tens), .seg(HEX5));
40     decoder_7seg m_units1 (.digit(m_units), .seg(HEX4));
41     decoder_7seg s_tens1 (.digit(s_tens), .seg(HEX3));
42     decoder_7seg s_units1 (.digit(s_units), .seg(HEX2));
43
44 endmodule

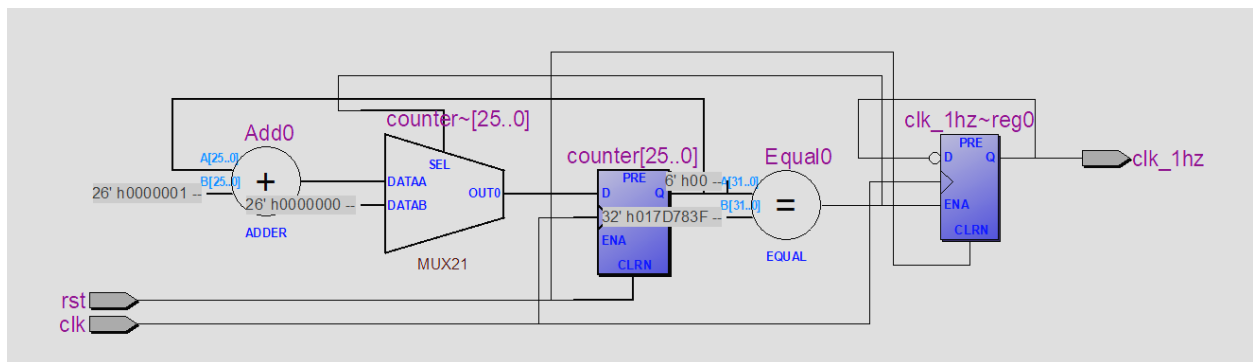
```

Hình 48: Top module bài 5

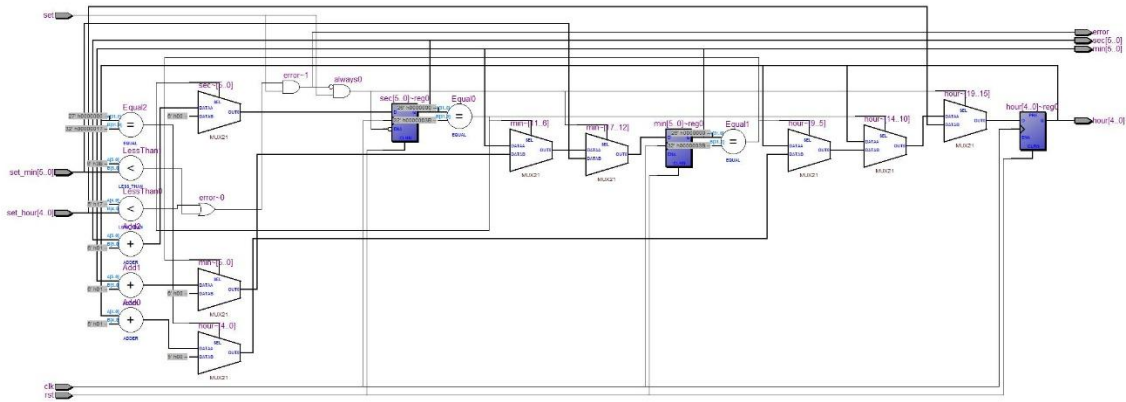
c. RTL view



Hình 49: RTL view top module

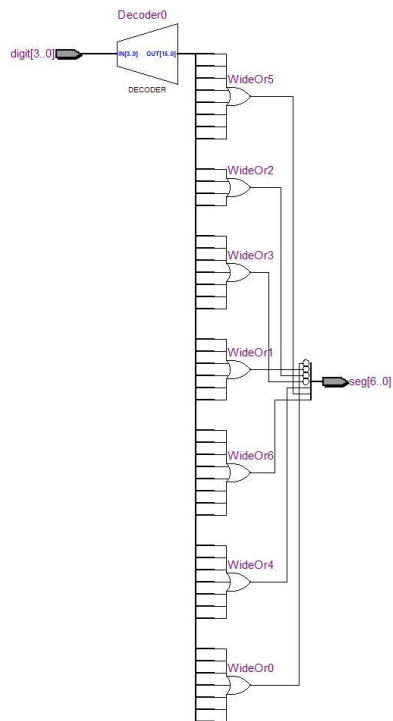


Hình 50: RTL khối clk_div



Hình 51: RTL khối time_counter

Trong module time_counter các tín hiệu hour, min, sec là các tín hiệu tuần tự và tín hiệu error là tín hiệu tổ hợp



Hình 52: RTL khối decoder_7seg